Software Architecture and Design Specification (SAD)

Peer-to-Peer Tutoring Scheduler

Project	Peer-to-Peer Tutoring Scheduler
Version	1.0
Authors	Sonia Sharma, P Saanvi, Kasissnu Ssinha, Kaustubh Akash
Date	27-10-2026
Status	Submitted for review

Revision history

Version	Date	Author	Change summary
1.0	27-10-2026	Team	Initial architecture, UML diagrams, API design and threat model

Approvals

Role	Name	Signature / Date
Course Coordinator		

1. Introduction

1.1 Purpose

This document specifies the architecture and detailed design of the Peer-to-Peer Tutoring Scheduler and explains the decisions behind them.

1.2 Scope

Covers the calendar web client, REST API services (auth, slots, bookings with conflict resolution, notifications, reviews), the background reminder worker, data storage, deployment and CI/CD.

1.3 Audience

Developers, QA engineers, the course instructor and evaluators, and maintainers.

1.4 Definitions

Slot, Booking, Conflict, Job, Worker, JWT, RBAC, ADR (Architecture Decision Record), STRIDE, TLS, ORM, CI/CD. See SRS section 1.4.

2. Document overview

2.1 How to use this document

Section 3 describes the architecture (components, pattern, technology, risks, security). Section 4 gives the design (sequence diagrams, APIs, error handling, UX).

2.2 Related documents

SRS v1.0 (requirements P2P-F, P2P-NF, P2P-SR), STP v1.0 (test plan), RTM (in SRS section 8).

3. Architecture

3.1 Goals and constraints

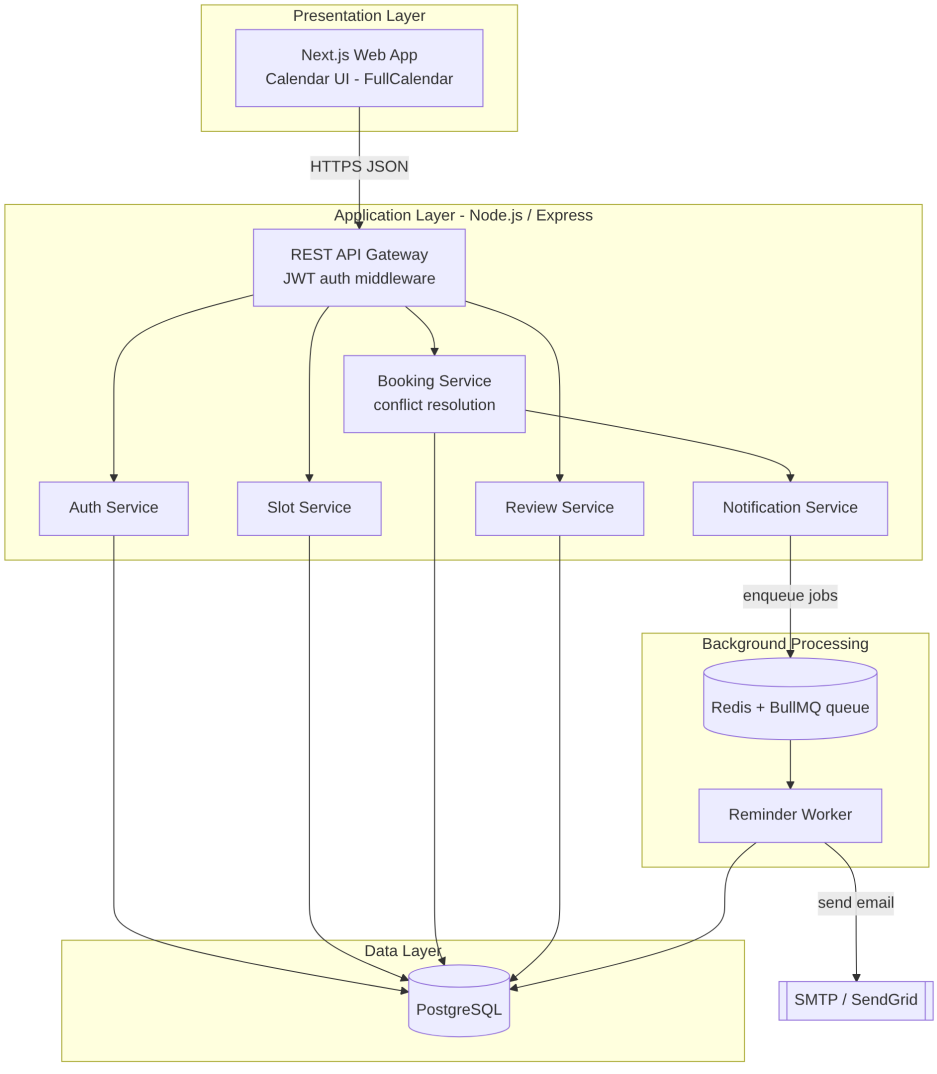
Goals: No double bookings under concurrency, reliable reminders, fast and friendly calendar experience, easy to test and deploy.

Constraints: DPDP Act 2023 principles for student personal data, OWASP security practice (TLS 1.2+, hashed passwords), dependence on an external email provider's limits, UTC time storage with local display, browser-only clients, and an open-source stack deployed as Docker containers.

3.2 Stakeholders and concerns

Stakeholder	Main concerns
Learners	Find a tutor quickly, never lose a booking, get reminded
Tutors	Easy slot management, no clashes, fewer no-shows
Admin / Instructor	Moderation, data privacy, demonstrable quality
Developers	Modularity, testability, simple local setup

3.3 Component (UML) diagram



Component diagram

3.4 Component descriptions

Component	Responsibility	Key requirements
Calendar UI (Next.js + FullCalendar)	Month/week/day views, filters, booking dialogs, local time display	F-009 to F-011
API Gateway (Express)	Routing, JWT verification, RBAC, validation, rate limiting	SR-003, SR-004, SR-005
Auth Service	Register, verify email, login, password hashing	F-001 to F-004, SR-002
Slot Service	Create, recur, edit and delete slots; prevent tutor overlap	F-005 to F-008
Booking Service	Reserve, cancel, reschedule; transactional conflict resolution; suggest alternatives	F-012 to F-018
Notification Service	Builds email content and enqueues confirmation and reminder jobs	F-019 to F-021
Reminder Worker (BullMQ)	Processes delayed jobs, sends email, retries with backoff	F-020 to F-022
Review Service	Ratings, reviews, average score	F-023
PostgreSQL	Users, slots, bookings, reviews, email logs	NF-003
Redis	Persistent job queue and rate-limit counters	NF-003, SR-004

Core data model: `users(id, name, email, password_hash, role, verified)`, `slots(id, tutor_id, subject, start_utc, end_utc, mode, status)`, `bookings(id, slot_id UNIQUE, learner_id, status, created_at)`, `reviews(id, booking_id UNIQUE, rating, comment)`, `email_logs(id, booking_id, type, status, attempts)`.

Conflict resolution strategy: (1) a `UNIQUE(slot_id)` constraint on active bookings, (2) `SELECT ... FOR UPDATE` row lock inside a transaction, and (3) an overlap query `start_utc < :end AND end_utc > :start` for the learner's existing bookings. Losers of a race receive `409 Conflict` with up to three alternative slots.

3.5 Chosen architecture pattern and rationale

Layered (n-tier) modular monolith

Layers (UI, API, service, data) give clear separation of concerns and make unit testing easy.

- The job queue decouples slow email sending from the request path (efficiency and reliability).
- **ADR-01:** Microservices rejected, as the operational overhead is too high for a two-person, two-sprint project; modules are kept separable for future split.
- **ADR-02:** PostgreSQL chosen over MongoDB because bookings need ACID transactions and row locking.
- **ADR-03:** BullMQ on Redis chosen over cron jobs because it supports delayed jobs, retries and persistence.

3.6 Technology stack and data stores

Layer	Technology
Frontend	Next.js 14 (React, App Router), FullCalendar, Tailwind CSS, Axios
Backend	Node.js 20, Express, Prisma ORM, Zod validation, JWT
Jobs	Redis 7, BullMQ, Nodemailer (SendGrid / Mailtrap sandbox)
Database	PostgreSQL 16
Testing	Jest, Supertest, React Testing Library, Playwright, k6
DevOps	Docker, Docker Compose, GitHub Actions, deployed on Render / Railway
Tracking	GitHub Projects (backlog, story points, sprints), Jira dashboard and burndown

CI/CD pipeline (GitHub Actions): on every PR, run install, lint, unit tests, integration tests (with Postgres and Redis service containers), build, and Docker image build. On merge to `main`, deploy to the staging environment and run smoke tests.

3.7 Risks and mitigations

Risk	Mitigation
Double booking under concurrent requests	DB transaction, row lock and unique constraint; concurrency test TC-BOOK-02
Reminder emails lost on server restart	Persistent Redis queue, idempotent jobs, retries
Time-zone mistakes	Store UTC, convert in UI, dedicated tests across zones
Email provider limits / spam folder	Sandbox in dev, verified sender domain, retry and logging
Tight sprint timeline	Prioritised backlog (High first), daily progress tracking, burndown chart

3.8 Traceability to requirements

Requirement	Component
P2P-F-001 to 004 (accounts)	Auth Service
P2P-F-005 to 008 (slots)	Slot Service
P2P-F-009 to 011 (calendar)	Calendar UI
P2P-F-012 to 018 (booking, conflicts)	Booking Service
P2P-F-019 to 022 (emails, jobs)	Notification Service, Reminder Worker
P2P-F-023 to 024 (reviews, admin)	Review Service, API Gateway

3.9 Security architecture (STRIDE threat model)

Threat	Example in this system	Mitigation
Spoofing	Logging in as another student	bcrypt passwords, verified institutional email, JWT with short expiry
Tampering	Changing `learnerId` in a request to cancel someone else's booking	Server-side ownership checks; user identity taken from token only
Repudiation	Tutor denies cancelling a session	Audit log of booking state changes with timestamp and user
Information disclosure	Exposing other students' emails	Public profile hides email; HTTPS; no personal data in logs
Denial of service	Bot books every slot	Rate limiting, per-learner booking cap, email verification
Elevation of privilege	Learner calls admin endpoints	RBAC middleware on every route, role stored server-side

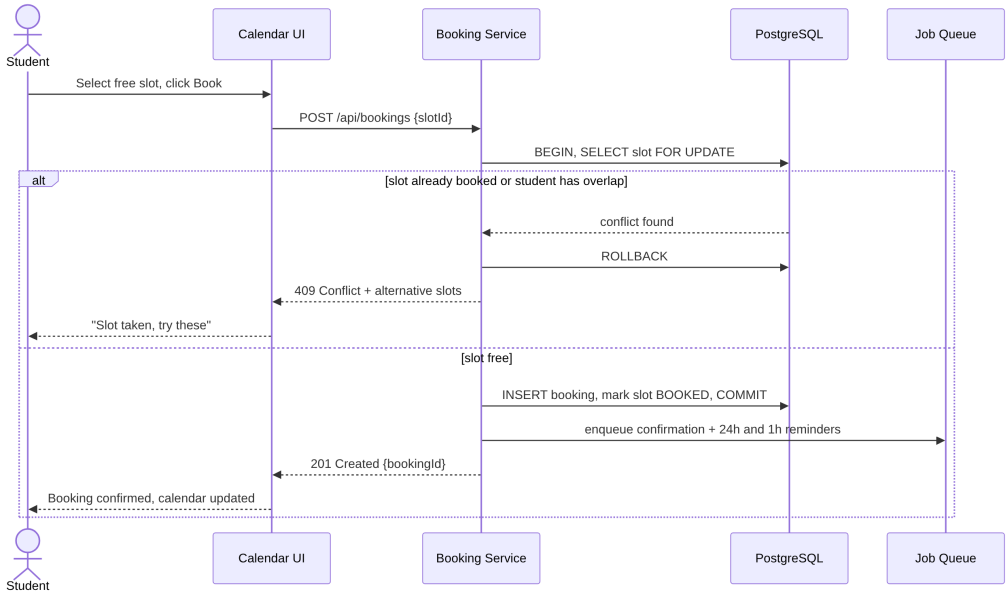
4. Design

4.1 Design overview

Each service exposes functions to an Express router; services contain business rules and use Prisma repositories for data access. Side effects (emails) are published as jobs, keeping request handlers short and testable.

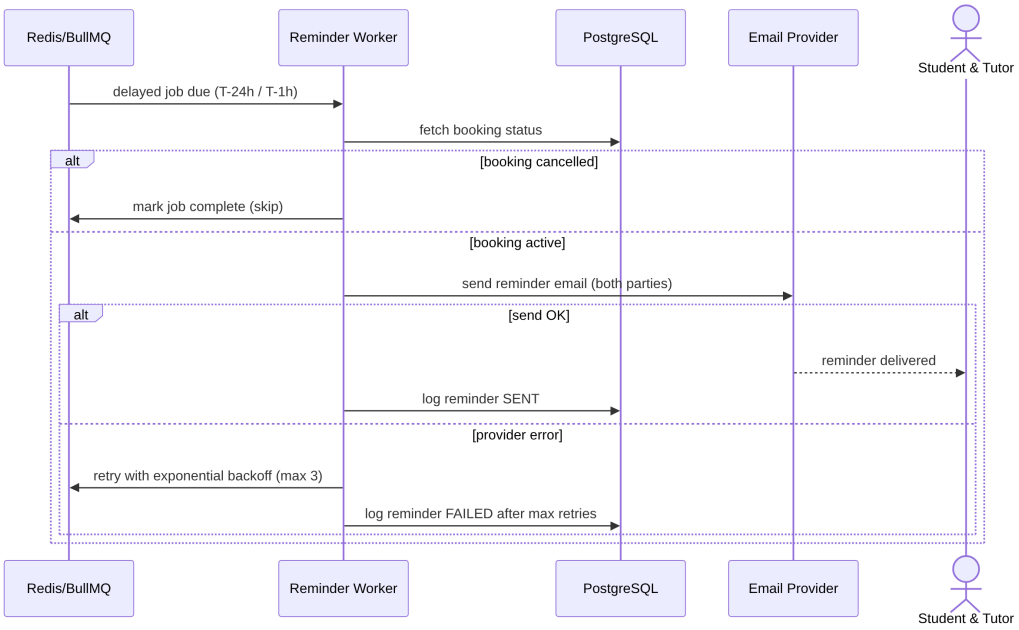
4.2 UML sequence diagrams

Sequence 1: Booking a slot with conflict resolution



Booking sequence

Sequence 2: Background email reminder with retry



Reminder sequence

4.3 API design

Component 1: Booking Service

Item	Detail
Endpoint	`POST /api/bookings`
Auth	Bearer JWT (role: Learner)
Request	`{ "slotId": "uuid", "note": "string (optional)" }`
Response 201	`{ "bookingId": "uuid", "slotId": "uuid", "start": "ISO-8601", "end": "ISO-8601", "status": "CONFIRMED" }`
Errors	400 invalid input, 401 not logged in, 403 own slot or unverified, 409 `{ "error": "SLOT_TAKEN" or "LEARNER_OVERLAP", "alternatives": [...] }`, 429 rate limit

Other endpoints: `DELETE /api/bookings/{id}` (cancel), `PATCH /api/bookings/{id} { newSlotId }` (reschedule), `GET /api/bookings/me`.

Component 2: Slot Service

Item	Detail
Endpoint	`POST /api/slots`
Auth	Bearer JWT (role: Tutor)
Request	`{ "subject": "DBMS", "start": "ISO-8601", "end": "ISO-8601", "mode": "ONLINE", "link": "url", "repeatWeeks": 0-8 }`
Response 201	`{ "created": [ { "slotId": "uuid", "start": "...", "end": "..." } ] }`
Errors	400 end before start or past date, 401, 403 not a tutor, 409 `TUTOR_OVERLAP`

Other endpoints: `GET /api/slots?subject=&tutor=&from=&to=&mode=` (browse), `PUT /api/slots/{id}`, `DELETE /api/slots/{id}`.

Component 3: Auth Service: `POST /api/auth/register`, `POST /api/auth/login` returns `{ token, user }`; errors 401 invalid credentials, 409 email exists.

4.4 Error handling, logging and monitoring

- Uniform error body `{ "error": CODE, "message": text, "details": {} }` produced by central Express error middleware.
- Structured JSON logs (pino) with request ID; passwords, tokens and emails are masked.
- Health endpoint `GET /api/health` checks DB and Redis.
- Monitoring metrics: API latency (p95), booking conflict rate, failed email job count, queue backlog; failed jobs visible in the BullMQ dashboard.

4.5 UX design

- Calendar is the home page; free slots in one colour with “Free” label, own bookings marked “Mine”.
- Clicking a slot opens a dialog with tutor, subject, time, mode and a single **Book** button.
- On conflict, the dialog explains why and shows alternative slots as one-click options.
- Times shown in the user’s local zone with zone label; mobile view defaults to day/list view.
- Accessible: keyboard navigable, WCAG AA contrast, ARIA labels on calendar events.

4.6 Open issues and next steps

Google Calendar sync, SMS/WhatsApp reminders, group sessions, waitlists for full tutors, and tutor no-show reporting are future enhancements. Deviations found during Sprint 1 and 2 will be documented and explained in the final demo.

5. Appendices

5.1 Glossary: see SRS section 1.4.

5.2 References: IEEE 42010 (architecture description), IEEE 830 (SRS), OWASP Top 10, Microsoft STRIDE.

5.3 Tools: Mermaid (diagrams), draw.io, Swagger / OpenAPI, GitHub Projects, Jira.